

Week 1 • Day 2

PostgreSQL Basics & Dataset Loading

Created the churn_db database, designed the customers table, and loaded all 7,043 rows from CSV

DATE	INTERN NAME	PROJECT	MENTOR
Day 2 of 28 Week 1		Customer Churn Prediction & LTV Engine	

■ Technologies Used Today

PostgreSQL pgAdmin 4 Python SQLAlchemy Pandas psycopg2 SQL

■ Today's Key Metrics

7,043

Rows Loaded

21

Table Columns

26.5%

Churn Rate Found

6

SQL Queries Run

■ Day Objectives

- Understand relational databases, tables, and SQL commands
- Create churn_db database in PostgreSQL using pgAdmin
- Design and create the customers table with correct schema
- Import telco_churn.csv into PostgreSQL using Python script
- Verify data load with SQL queries in pgAdmin

■ Tasks Completed Today

1. Opened pgAdmin 4 and connected to PostgreSQL server with superuser password
2. Created churn_db database via pgAdmin GUI (right-click Databases → Create)

3. Opened Query Tool for churn_db and wrote CREATE TABLE statement with 21 columns
4. Defined correct data types for each column: VARCHAR for text, INTEGER for tenure, NUMERIC for charges
5. Created src/db/load_data.py — Python script to load CSV into PostgreSQL using Pandas + SQLAlchemy
6. Wrote get_engine() function using SQLAlchemy create_engine() with .env credentials
7. Ran load_data.py — successfully loaded 7,043 rows into customers table
8. Verified row count: SELECT COUNT(*) returned 7,043
9. Wrote 6 practice SQL queries: COUNT, GROUP BY, AVG, DISTINCT, ORDER BY, LIMIT
10. Discovered churn distribution: 5,174 No (73.5%) vs 1,869 Yes (26.5%)
11. Committed all new files to GitHub

■ Concepts Learned

- Relational database structure: tables, rows, columns, primary keys
- SQL data types: VARCHAR, INTEGER, NUMERIC — when to use each
- What SQLAlchemy Engine is and why it manages database connections
- The difference between NaN (Python null) and empty string in SQL
- What 'class imbalance' means: 73.5% vs 26.5% — critical for ML later
- GROUP BY: how to aggregate data by category in SQL
- Why TotalCharges was stored as VARCHAR (has hidden empty strings)

■ Tools / Software Used

Tool / Library	Version / Notes	Status
psycopg2-binary	2.9.9	✓ Already installed
SQLAlchemy	2.0.23	✓ Already installed
python-dotenv	1.0.0	✓ Already installed
pgAdmin 4	Existing install	✓ Connected to DB

■ Outputs / Deliverables Produced

- ◆ churn_db database created in PostgreSQL
- ◆ customers table created with 21 columns and correct schema
- ◆ 7,043 rows successfully loaded from telco_churn.csv
- ◆ src/db/load_data.py — reusable data loader script
- ◆ SQL query results: churn rate = 26.5%, avg monthly charge = \$64.76
- ◆ GitHub commit with load_data.py and SQL schema

■ Problems Encountered & Solutions

Problem Faced	How I Solved It
psycopg2.OperationalError: could not connect to server	Opened Windows Services, found postgresql-x64-15, right-clicked → Start
TotalCharges column had mixed types (numbers + empty strings)	Defined TotalCharges as VARCHAR(20) in the table schema to accept both — will convert to numeric in Day 6

■ GitHub Commit Record

Commit Message	Week1-Day2: PostgreSQL setup, created customers table, loaded 7043 rows
Branch	main
Files Changed	src/db/load_data.py
Commit Hash	(fill after push)

■ End-of-Day Submission Checklist

■ churn_db database created	■ customers table has 21 columns
■ 7,043 rows loaded from CSV	■ COUNT(*) returns 7043
■ Churn distribution verified	■ 6 SQL queries executed
■ load_data.py committed	■ pgAdmin shows data in grid
■ No import errors in Python	■ Day 2 commit on GitHub

■ Self Assessment

8/10 Self Rating	6 Hours Spent	Week 1, Day 2 Progress	2/28 Day in Program
---------------------	------------------	---------------------------------	------------------------

■ Key Learnings & Reflections

★ Month-to-month customers churn at 42.7% — immediately the strongest signal found

- ★ Always verify row counts after data loading — don't assume it worked
- ★ SQLAlchemy's `if_exists='replace'` is useful in development but dangerous in production
- ★ Empty strings are NOT the same as NULL in SQL — must check both types of missing data

■ Plan for Tomorrow

- Learn Pandas DataFrame fundamentals: `head()`, `info()`, `describe()`, `dtypes`
- Create `src/db/connection.py` — centralised database connector module
- Load dataset from PostgreSQL into Pandas and run basic analysis
- Confirm churn rate and contract type distribution via Python

INTERN SIGNATURE	MENTOR SIGNATURE	DATE SUBMITTED
_____	_____	_____